EAI BDCC 2026

Original submission / revised paper

Marked comparison — for review, not conference upload.

Left: original submission (21 pages).

Right: revised camera-ready manuscript (23 pages).

Red shading and strike-through: deleted or replaced words.

Blue shading and underline: added or replacement words.

Colored figure frames: removed/replaced or added figure assets.

Words are matched across the complete documents.

Pages are paired by page number; passages can move between pages.

Relocated wording may appear as a deletion and an addition.

Running heads and line-break hyphenation are ignored.

Equations, tables, figures and page layouts retain their PDF appearance.

Use the separate clean camera-ready PDF for submission.

Original SHA-256: 2c21a41ed40d3ecea0c99bba

Revised SHA-256: 1c6deb6ac180507b8c350738

Semantic Structure and Revision in Temporal Evidence Graphs for Generative Monitoring

Anonymous Authors

Anonymous Institution

Abstract. Generated explanations can remain convincing after their evidence changes. A corrected sensor measurement, for example, can invalidate an LLM's comparison between two time windows. We present a temporal evidence graph framework that uses Neo4j to keep generated explanations traceable and current. The LLM proposes claims with evidence references and dependencies, which are checked and stored alongside versioned measurements. When evidence changes, graph traversal identifies claims to recheck. Unsupported claims are withdrawn, while valid alternative support preserves justified conclusions. A network dashboard makes these decisions inspectable. Two conditional results identify potentially affected claims and explain when direct checking misses necessary updates. We measure this gap through transitive exposure. Experiments use synthetic streams, WESAD and PPG-DaLiA recordings, with 240 controlled structural cases. In GPU-generated program replays, direct maintenance misses 183 of 1,210 required changes. Full maintenance misses none and withdraws no unaffected claims. Exposure predicts misses in all 860 eligible replay groups. Checked displays achieve 100% source grounding under the numerical protocol and 75.5–78.2% required-fact recall. Neo4j and a relational control produce matching outcomes. The framework connects generative AI, graph storage and explicit support rules to make explanation revision selective and inspectable.

Keywords: Temporal knowledge graphs · Evidence provenance · Generative explanations · Incremental maintenance · Physiological streams

1 Introduction

Generative AI can turn complex sensor data into explanations that people can understand and inspect. Its value in continuous monitoring depends on keeping those explanations connected to the evidence as measurements arrive, change or are corrected. A useful monitoring system should identify which statements need updating, preserve information that remains valid and make the reasons for these decisions visible to the user.

Consider an LLM that compares two sensor windows and reports a change in skin conductance. A later correction to one measurement may invalidate the reported difference and other statements that depend on it. Statements supported by unaffected measurements can remain valid. Recording these relationships

Semantic Structure and Revision in Temporal Evidence Graphs for Generative Monitoring

John Z Willis[1], Alexander V. Mantzaris[1,*], and James H. Korris[2]

[1] Dr. Yvette Kanouff School of Data, Mathematical, and Statistical Sciences, University of Central Florida, Orlando, FL 32816, USA
john.willis@ucf.edu, alexander.mantzaris@ucf.edu
[*] Corresponding author
[2] Creative Technologies Inc., Hollywood, CA 90028-6039, USA
james.korris@cretecinc.com

Abstract. Generated explanations can remain convincing after their evidence changes. A corrected sensor measurement, for example, can invalidate an LLM's comparison between two time windows. We present a temporal evidence graph framework that uses Neo4j to link bounded numerical claims to their evidence and revision history. The LLM proposes claims with evidence references and dependencies, which are checked and stored alongside versioned measurements. When evidence changes, graph traversal identifies claims to recheck. Unsupported claims are withdrawn, while valid alternative support preserves justified conclusions. A network dashboard displays these connections. Two standard conditional maintenance properties identify potentially affected claims and explain when direct checking misses necessary updates. Transitive exposure is a retrospective diagnostic of this gap. Experiments use synthetic streams, WESAD and PPG-DaLiA recordings, with 240 controlled structural cases. Under corrected admission, direct checking misses 208 of 1,294 required changes in controlled generated-program replays; full maintenance misses none. Ordinary task-guided generation has zero exposure in all 360 revision batches. Admission trades required-fact recall for grounding under a shared numerical contract. Exhaustive rechecking and archived fresh answers provide additional baselines. Neo4j and a relational control produce matching outcomes. The results concern bounded numerical revision; they do not establish LLM utility, database performance superiority or benefits to human reviewers.

Keywords: Temporal knowledge graphs · Evidence provenance · Generative explanations · Incremental maintenance · Physiological streams

1 Introduction

Generative AI can turn complex sensor data into explanations that people can understand and inspect. Its value in continuous monitoring depends on keeping those explanations connected to the evidence as measurements arrive, change or are corrected. A useful monitoring system should identify which statements

makes it possible to revise an explanation selectively and show the user how the correction affects the answer.

We present a temporal evidence graph framework that combines LLM-generated explanations with a Neo4j graph database and an interactive review dashboard. The LLM receives features extracted from sensor streams and produces readable statements accompanied by evidence references and declared dependencies. The database connects these claims to their supporting measurements and preserves earlier versions. When evidence changes, graph traversal identifies the claims to recheck. The system withdraws unsupported statements and retains those justified by surviving evidence. Labeled network views let users follow these connections from an explanation back to its sources.

Our central contribution connects this practical workflow to a formal account of when revision is necessary. We model how several inputs can jointly support a claim and how an alternative evidence route can preserve it. Two conditional results identify the region of the graph that a correction can affect and establish when checking only immediate dependencies will miss required changes. We introduce transitive exposure to measure this gap, connecting the graph's structure to an observable maintenance outcome.

Experiments use synthetic streams and two real datasets, WESAD and PPG-DaLiA. Across 240 controlled structural cases, we evaluate exact programs and dependencies actually produced by the LLM. Full propagation handles all 1,210 required claim changes in generated-program replays without withdrawing unaffected claims. Direct checking misses 183 of these changes. Predicted and observed misses agree in all 860 eligible replays, and Neo4j produces matching maintenance outcomes with an indexed relational implementation. The waveform experiments also identify settings where direct checking suffices, establishing when propagation provides additional value.

Independent evaluation finds 100% grounding among checked displayed claims under the numerical protocol, with subject-averaged required-fact recall from 75.5% to 78.2%. Measuring both properties shows how successfully the system preserves supported content and covers the requested information. Together, the theory, experiments and network interface provide an auditable approach to maintaining generative explanations as their evidence evolves.

1.1 Relation to Prior Work

Truth-maintenance systems revise justified beliefs when their premises change [1]. Incremental view maintenance updates database results when their inputs change [4]. Provenance semirings distinguish jointly required inputs from alternative derivations [3]. We connect these established ideas to the dependencies a bounded generator actually expresses and to independently measured maintenance outcomes.

PROV-DM distinguishes entities, activities, derivation, revision and attribution [13]. It informs our separation of recording provenance, generated assertions and review events. We implement a property graph with Python admission checks and database identity constraints. GraphRAG builds entity graphs and

need updating, preserve information that remains valid and make the reasons for these decisions visible to the user.

Consider an LLM that compares two sensor windows and reports a change in skin conductance. A later correction to one measurement may invalidate the reported difference and other statements that depend on it. Statements supported by unaffected measurements can remain valid. Recording these relationships makes it possible to revise an explanation selectively and show the user how the correction affects the answer.

We present a temporal evidence graph framework that combines LLM-generated explanations with a Neo4j graph database and an interactive review dashboard. The LLM receives features extracted from sensor streams and produces readable statements accompanied by evidence references and declared dependencies. The database connects these claims to their supporting measurements and preserves earlier versions. When evidence changes, graph traversal identifies the claims to recheck. The system withdraws unsupported statements and retains those justified by surviving evidence. Labeled network views let users follow these connections from an explanation back to its sources.

Our contribution connects this practical workflow to established conditions for justified-belief and dependency maintenance. We model how several inputs can jointly support a claim and how an alternative evidence route can preserve it. Two standard conditional properties identify the region of the graph that a correction can affect and establish when checking only immediate dependencies will miss required changes. We use transitive exposure to describe the observed gap retrospectively, not to predict it from graph structure alone.

Experiments use synthetic streams and two real datasets, WESAD and PPG-DaLiA. Across 240 controlled structural cases, we evaluate exact programs and dependencies actually produced by the LLM. These controlled replays are not a deployment estimate. A relational control tests storage equivalence; the waveform experiments also identify settings where direct checking suffices.

Admission trades required-fact recall for grounding under a bounded numerical contract. Here, "semantic structure" and "explanations" refer to typed statistics and declared support rules, not unrestricted physiological interpretation. The generator proposes phrasing, citations and groupings; its utility over a deterministic template and the dashboard's benefit to human reviewers have not been established.

1.1 Relation to Prior Work

Truth-maintenance systems revise justified beliefs when their premises change [6]. Incremental view maintenance updates database results when their inputs change [11]. Provenance semirings distinguish jointly required inputs from alternative derivations [10]. We connect these established ideas to the dependencies a bounded generator actually expresses and to independently measured maintenance outcomes.

PROV-DM distinguishes entities, activities, derivation, revision and attribution [24]. It informs our separation of recording provenance, generated assertions

community summaries for corpus-level question answering [2]. Graphiti, described in the Zep architecture, maintains temporal information for agent memory [6]. Our unit of evaluation is a revisable numerical proposition and its admissible supporting inputs. We measure whether evidence grounds that proposition. This differs from asking whether a model's account of its own computation faithfully describes its decision process [11].

2 A Typed Temporal Semantic Representation

2.1 Schema-Guided Ontology Instantiation

The ontology is fixed in code and specifies the allowed objects and relationships. The generator proposes instances within this vocabulary. Formally, let

$$\begin{aligned} \Omega &= (\mathcal{T}, \mathcal{R}, \mathrm{dom}, \mathrm{ran}, \mathcal{C}), \\ G_t &= (V_t, E_t, \vartheta_V, \vartheta_E, a_t). \end{aligned} \tag{1}$$

Here $\mathcal{T}$ and $\mathcal{R}$ are the node and relation vocabularies. $\vartheta_V : V_t \to \mathcal{T}$ and $\vartheta_E : E_t \to \mathcal{R}$ assign their types. An edge of type r connects allowed types in $\mathrm{dom}(r)$ and $\mathrm{ran}(r)$. Attributes a_t record immutable and logical IDs, dataset, subject, session, event interval, ingestion time, version, value, unit and evidence state. Constraints $\mathcal{C}$ require immutable identifiers, finite acyclic declared dependencies and the admission checks below. Ingestion time τ_v is distinct from the type maps ϑ. At knowledge time t, only records with $\tau_v \le t$ are visible. Assessments become visible at their recorded knowledge time. Queries select the greatest version visible at their knowledge time, using ingestion time and ID to break ties. Historical queries retain that time. Tied assessments retain their trigger IDs. Conflicting states remain ambiguous because the timestamp does not establish the order of triggers.

Table 1 lists the stored objects. Observation windows carry hashes, sample indices and channel metadata, while raw arrays remain outside the database. Features carry units, extractor metadata and source IDs. The `with_entities` function adds subject and channel identities. The model proposes proposition fields, evidence references, parent links and sentences. Claims that pass the bounded checks are inserted immutably. The Neo4j adapter adds `SupportAssessment` nodes and `ASSESSED_AS` edges to the storage totals. Inserting an edge can create a placeholder for an unresolved source Record. None occurs in the original exports. Table 2 gives relation meanings and creation rules. Neo4j enforces unique scope/ID pairs. Python checks record kinds, immutable payloads, acyclicity and numerical admission. The database itself does not enforce these semantic checks.

Construction first normalizes the available evidence, then checks the model's proposals and inserts the accepted records:

$$G_t = \Gamma_\Omega(E_{\le t}, \widehat{C}_{\le t}) = \mathrm{Insert}_{\mathrm{immutable}}\Big(\mathrm{Check}_q\Big(\mathrm{Normalize}_\Omega(E_{\le t}), \widehat{C}_{\le t}\Big)\Big). \tag{2}$$

$E_{\le t}$ is the observation and revision log. $\widehat{C}_{\le t}$ contains proposals generated from the normalized visible evidence. Rejected proposals remain in the request archive. New

and review events. We implement a property graph with Python admission checks and database identity constraints. GraphRAG builds entity graphs and community summaries for corpus-level question answering [7]. Graphiti, described in the Zep architecture, maintains temporal information for agent memory [17]. Our unit of evaluation is a revisable numerical proposition and its admissible supporting inputs. We measure whether evidence grounds that proposition. This differs from asking whether a model's account of its own computation faithfully describes its decision process [22]. Zep/Graphiti uses temporal relation intervals and LLM-based conflict resolution; our immutable numerical citations and executable AND/OR checks determine support. This is an architectural comparison, not an empirical head-to-head benchmark.

Alternative justifications are explicit in assumption-based TMS [4], while AGM characterizes rational belief revision [1]. Our conditions apply established maintenance ideas, including incremental Datalog [14]. Event and knowledge time follow the valid/transaction-time distinction [12]; named graphs also support provenance descriptions [3]. AIS and ALCE assess attribution and citation quality [16,9]; data-to-document work tests faithful expression of numerical records [25]. Chain-of-Verification and Self-RAG verify or critique generations [5,2], whereas maintenance revisits admitted claims after source changes. SensorLLM addresses sensor-language alignment for activity recognition [13]; PhysioLLM studies interactive wearable-data interpretation [8]. Our fixed numerical revision task is narrower.

2 A Typed Temporal Semantic Representation

2.1 Schema-Guided Ontology Instantiation

The ontology is fixed in code and specifies the allowed objects and relationships. The generator proposes instances within this vocabulary. Formally, let

$$\begin{aligned} \Omega &= (\mathcal{T}, \mathcal{R}, \mathrm{dom}, \mathrm{ran}, \mathcal{C}), \\ G_t &= (V_t, E_t, \vartheta_V, \vartheta_E, a_t). \end{aligned} \tag{1}$$

Here V_t and E_t are visible nodes and edges; a_t maps them to attributes. $\mathcal{T}$ and $\mathcal{R}$ are the node and relation vocabularies. $\vartheta_V : V_t \to \mathcal{T}$ and $\vartheta_E : E_t \to \mathcal{R}$ assign their types. An edge of type r connects allowed types in $\mathrm{dom}(r)$ and $\mathrm{ran}(r)$. Attributes a_t record immutable and logical IDs, dataset, subject, session, event interval, ingestion time, version, value, unit and evidence state. The integrity-constraint set $\mathcal{C}$ requires immutable identifiers, finite acyclic declared dependencies and the admission checks below. Ingestion time τ_v records when node v becomes known. At knowledge time t, only records with $\tau_v \le t$ are visible. Assessments become visible at their recorded knowledge time. Queries select the greatest version visible at their knowledge time, using ingestion time and ID to break ties. Historical queries retain that time. Tied assessments retain their trigger IDs. Conflicting states remain ambiguous because the timestamp does not establish the order of triggers.

Table 1. Implemented object vocabulary. Checked status records an admission or assessment operation. It does not establish physiological truth.

Stored object	Meaning	Construction
Subject / Sensor	Identity and channel	deterministic
Observation	Hashed sample window	deterministic
FeatureVersion	Value or availability watch	deterministic
ClaimVersion	Bounded proposition, references	model; checked
ExplanationVersion	Text and display membership	administrative
ReviewEvent	Recorded review action	administrative
SupportAssessment	State, time and trigger	evaluation
Record placeholder	Unresolved referenced ID	administrative

Table 2. Relation construction and direction. R denotes a Record, P an unresolved placeholder, F a feature, C a claim, O an observation, X an explanation and A an assessment. The adapter constructs edges. Admission checks give them claim-specific semantics. Administrative relations do not express entailment.

Relation	Domain → range	Meaning and creation rule
`DEPENDS_ON`	R → R/P	Each source ID: deterministic input, model citation or declared parent.
`DERIVED_FROM`	F → R/P	Feature source IDs: extraction provenance.
`SUPPORTS`	R/P → C	Reverse source edge on admission; validity at insertion only.
`SUPERSEDES`	R → R/P	New version to its predecessor ID.
`CONTAINS`	X → R/P	Explanation source IDs: display membership; also dependencies.
`ASSESSED_AS`	R → A	Outcome keyed by scope, record, knowledge time and trigger.
`CONTRADICTS`	R → R	Trigger to adversely assessed record; time from assessment.
`FOR_SUBJECT`	R → subject	Subject membership from deterministic identities.
`OBSERVED_BY`	O → sensor	Channel identity from recording metadata.
`REVIEWED_BY_EVENT`	R/P → review	Reviewed item to appended review event.

Table 1 lists the stored objects. Observation windows carry hashes, sample indices and channel metadata, while raw arrays remain outside the database. Features carry units, extractor metadata and source IDs. The `with_entities` function adds subject and channel identities. The model proposes proposition fields, evidence references, parent links and sentences. Claims that pass the bounded checks are inserted immutably. The Neo4j adapter adds `SupportAssessment` nodes and `ASSESSED_AS` edges to the storage totals. Inserting an edge can create a placeholder for an unresolved source Record. None occurs in the original exports. Table 2 gives relation meanings and creation rules. Neo4j enforces unique scope/ID pairs. Python checks record kinds, immutable payloads, acyclicity and numerical admission. The database itself does not enforce these semantic checks. Neo4j was selected for typed-edge traversal and network exports; recursive SQL expresses the same dependencies. Logical parity does not establish an advantage in database performance.

Table 1. Implemented object vocabulary. Checked status records an admission or assessment operation. It does not establish physiological truth.

Stored object	Meaning	Construction
Subject / Sensor	Identity and channel	deterministic
Observation	Hashed sample window	deterministic
FeatureVersion	Value or availability watch	deterministic
ClaimVersion	Bounded proposition, references	model; checked
ExplanationVersion	Text and display membership	administrative
ReviewEvent	Recorded review action	administrative
SupportAssessment	State, time and trigger	evaluation
Record placeholder	Unresolved referenced ID	administrative

Construction first normalizes the available evidence, then checks the model's proposals and inserts the accepted records:

$$G_t = \Gamma_\Omega(L_{\le t}, \widehat{C}_{\le t}) = \mathrm{Insert}_{\mathrm{immutable}}\Big(\mathrm{Check}_q\Big(\mathrm{Normalize}_\Omega(L_{\le t}), \widehat{C}_{\le t}\Big)\Big)\,. \quad (2)$$

$L_{\le t}$ is the observation and revision log visible by t (distinct from edge set E_t); q specifies the query scope and Γ_Ω the schema-constrained construction operator. $\widehat{C}_{\le t}$ contains proposals generated from the normalized visible evidence. Rejected proposals remain in the request archive. New assessments and explanation versions extend the graph without overwriting earlier content. The process instantiates the specified ontology rather than discovering one.

These rules explain much of the graph's topology. Each `source_ids` entry creates a `DEPENDS_ON` edge from dependent to input. Features also receive `DERIVED_FROM` edges, and accepted claims receive `SUPPORTS` edges in the opposite direction. Display membership creates both `CONTAINS` and `DEPENDS_ON` edges. Subject membership creates `FOR_SUBJECT` hubs. Our analysis separates these administrative, provenance and reasoning roles.

assessments and explanation versions extend the graph without overwriting earlier content. The process instantiates the specified ontology rather than discovering one.

These rules explain much of the graph's topology. Each `source_ids` entry creates a `DEPENDS_ON` edge from dependent to input. Features also receive `DERIVED_FROM` edges, and accepted claims receive `SUPPORTS` edges in the opposite direction. Display membership creates both `CONTAINS` and `DEPENDS_ON` edges. Subject membership creates `FOR_SUBJECT` hubs. Our analysis separates these administrative, provenance and reasoning roles.

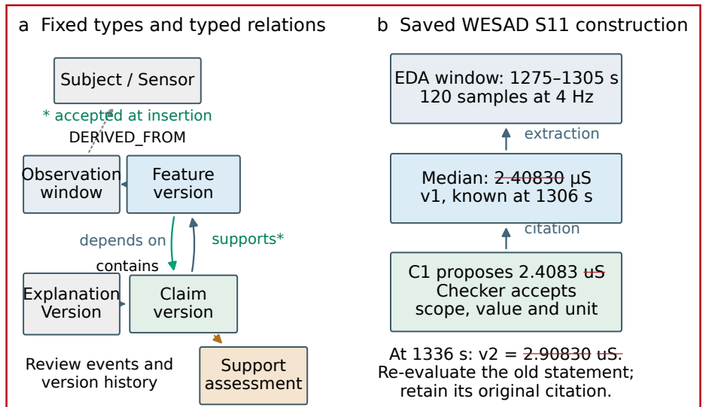


Fig. 1. Fixed vocabulary and an actual instance. Blue denotes deterministic recording and feature construction, green a checked model proposition, grey identity or display administration, and amber assessment or version effects. The concrete path comes from the exported WESAD S11 correction scope. Its source hash, sample indices and full IDs accompany the figure. Arrows preserve stored direction. The schema shown is a readable subset of the complete construction table.

In the saved WESAD S11 episode (Figure 1), the electrodermal activity (EDA) window at 1275–1305 s produces a median of 2.408303 μS. The model proposes 2.408303 μS and cites that feature. The claim passes the bounded checks. Insertion records its `DEPENDS_ON` citation and reverse `SUPPORTS` edge. A later feature version changes the median to 2.908303 μS, while the observation hash and original claim citation remain unchanged. Re-evaluation changes the claim's current support without rewriting its history.

Table 2. Relation construction and direction. R denotes a Record, P an unresolved placeholder, F a feature, C a claim, O an observation, X an explanation and A an assessment. The adapter constructs edges. Admission checks give them claim-specific semantics. Administrative relations do not express entailment.

Relation	Domain → range	Meaning and creation rule
`DEPENDS_ON`	R → R/P	Each source ID: deterministic input, model citation or declared parent.
`DERIVED_FROM`	F → R/P	Feature source IDs: extraction provenance.
`SUPPORTS`	R/P → C	Reverse source edge on admission; validity at insertion only.
`SUPERSEDES`	R → R/P	New version to its predecessor ID.
`CONTAINS`	X → R/P	Explanation source IDs: display membership; also dependencies.
`ASSESSED_AS`	R → A	Outcome keyed by scope, record, knowledge time and trigger.
`CONTRADICTS`	R → R	Trigger to adversely assessed record; time from assessment.
`FOR_SUBJECT`	R → subject	Subject membership from deterministic identities.
`OBSERVED_BY`	O → sensor	Channel identity from recording metadata.
`REVIEWED_BY_EVENT`	R/P → review	Reviewed item to appended review event.

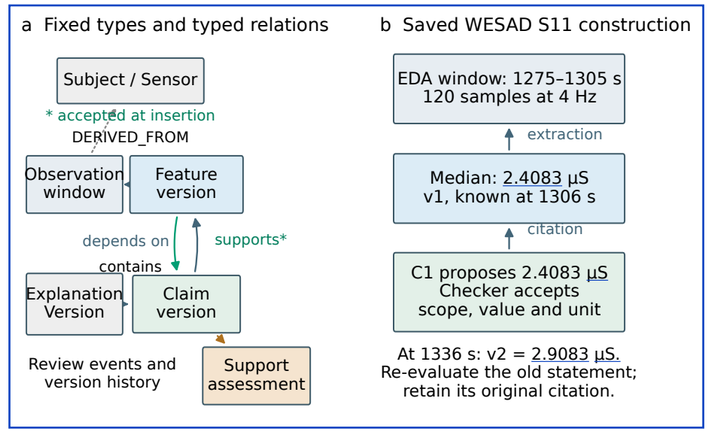


Fig. 1. Fixed vocabulary and an actual instance. Blue denotes deterministic recording and feature construction, green a checked model proposition, grey identity or display administration, and amber assessment or version effects. The concrete path comes from the exported WESAD S11 correction scope. Its source hash, sample indices and full IDs accompany the figure. Arrows preserve stored direction. The schema shown is a readable subset of the complete construction table.

2.2 Edges as Explicit Semantic Commitments

Edge types explain the declared justification. Citations offer evidence, parent links declare prerequisites, and extraction edges record feature origins. Membership and co-occurrence imply neither support nor dependence. An LLM-proposed `DEPENDS_ON` edge does not establish necessity. `SUPPORTS` records acceptance at insertion, while `CONTRADICTS` records a revision-triggered adverse assessment with an associated knowledge time. Exports preserve these origins and distinguish proposed, accepted and displayed links.

2.3 The Query-Specific Semantic Core

The semantic core selects an answer's claims and cited inputs for inspection and revision:

$$K_{q,t} = \pi_{q,\Omega}(G_t), \qquad V(K_{q,t}) = C^*_{q,t} \cup F_{q,t}, \quad E(K_{q,t}) = D_{q,t} \cup P_{q,t}. \tag{3}$$

$C^*_{q,t}$ contains the latest displayed claims and their declared claim ancestors. $F_{q,t}$ contains cited visible features and explicit availability watches. $D_{q,t}$ contains their stored dependency edges. When a cited feature has a newer visible version, $P_{q,t}$ traces the version chain, retaining citation and supersession IDs. These labelled paths are derived views, not directly stored Neo4j relationships.

Linked metadata preserves subject and sensor identities, original sentences, versions, explanation containers and timed assessments. Observation provenance can be expanded. Each claim's asserted fields determine its scope.

The complete graph contains all stored records. The active view contains the latest evidence and current displays across queries, with observation provenance. The core focuses on one answer. It reveals which windows and operands support a statement, which versions were known, and whether support survives a correction. We check these questions against immutable input logs or exact program specifications, independently of database support flags.

2.4 Producing the Human-Review Network View

The review view traces an answer to its evidence by expanding the semantic core with original citations, observation provenance and visible newer versions. For node budget B and recorded times T,

$$H_{q,t} = \mathrm{View}_B(G_t, q), \quad p = \mathrm{Layout}\left(\bigcup_{u \in T} H_{q,u}; m\right), \quad I_{q,t} = \mathrm{Render}(H_{q,t}, p). \tag{4}$$

H is the bounded expansion, p stores coordinates and edge routes, and I is the drawing. Mode m selects deterministic Graphviz `dot` or seeded spring positioning. The spring layout uses an undirected projection for placement only. Analysis and arrows retain the directed, typed relationships. Time comparisons share coordinates, while records not yet ingested remain hidden. Layout changes

In the saved WESAD S11 episode (Figure 1), the electrodermal activity (EDA) window at 1275–1305 s produces a median of 2.408303 μS. The model proposes 2.408303 μS and cites that feature. The claim passes the bounded checks. Insertion records its `DEPENDS_ON` citation and reverse `SUPPORTS` edge. A later feature version changes the median to 2.908303 μS, while the observation hash and original claim citation remain unchanged. Re-evaluation changes the claim's current support without rewriting its history.

2.2 Edges as Explicit Semantic Commitments

Edge types explain the declared justification. Citations offer evidence, parent links declare prerequisites, and extraction edges record feature origins. Membership and co-occurrence imply neither support nor dependence. An LLM-proposed `DEPENDS_ON` edge does not establish necessity. `SUPPORTS` records acceptance at insertion, while `CONTRADICTS` records a revision-triggered adverse assessment with an associated knowledge time. Exports preserve these origins and distinguish proposed, accepted and displayed links.

2.3 The Query-Specific Semantic Core

The semantic core selects an answer's claims and cited inputs for inspection and revision:

$$K_{q,t} = \pi_{q,\Omega}(G_t), \qquad V(K_{q,t}) = C^*_{q,t} \cup F_{q,t}, \quad E(K_{q,t}) = D_{q,t} \cup P_{q,t}. \tag{3}$$

$C^*_{q,t}$ contains the latest displayed claims and their declared claim ancestors. $F_{q,t}$ contains cited visible features and explicit availability watches. $D_{q,t}$ contains their stored dependency edges. When a cited feature has a newer visible version, $P_{q,t}$ traces the version chain, retaining citation and supersession IDs. These labelled paths are derived views, not directly stored Neo4j relationships.

Linked metadata preserves subject and sensor identities, original sentences, versions, explanation containers and timed assessments. Observation provenance can be expanded. Each claim's asserted fields determine its scope.

The complete graph contains all stored records. The active view contains the latest evidence and current displays across queries, with observation provenance. The core focuses on one answer. It reveals which windows and operands support a statement, which versions were known, and whether support survives a correction. We check these questions against immutable input logs or exact program specifications, independently of database support flags.

2.4 Producing the Human-Review Network View

The review view traces an answer to its evidence by expanding the semantic core with original citations, observation provenance and visible newer versions. For

presentation only. Distances represent neither semantic similarity nor neural state.

The implementation follows declared inputs within the selected database scope. Ingestion and assessment times control visibility, while event intervals specify when measurements or assertions apply. Original `DEPENDS_ON` citations remain intact. Derived paths trace newer versions by following supersession edges in reverse. Availability, support and display membership remain separate. Of 432 assessment groups sharing a record and time, 245 contain conflicting ~~states. These remain visibly ambiguous, with outcomes~~ and trigger ~~IDs preserved.~~ Display membership follows the saved explanation ~~version. Evaluation uses independent reference truth.~~

The node budget favors complete alternative support routes. Omissions or incomplete groups trigger a *partial view* warning and expansion control. Identity hubs, display records, assessments and reverse `SUPPORTS` links remain accessible in details. Structured views place jointly required features in labelled AND enclosures. Each row maps to a stored ID, and dashed bundles identify the dependency edges they summarize. Different groups provide alternative support routes.

Green, blue and grey identify claims, features and observations. Outlines and text show revisions and support status. Arrows retain their stored direction and labels or identify grouped edges. Figures 2 and 3 share typed JSON views, ID maps and rendering with the Streamlit dashboard. Its *Classic network* mode shows individual nodes (Figure 4). Controls select case, time, focus, route and record. Details expose full statements, exact values, units, intervals, versions, source IDs and assessments. Highlighting changes appearance only, expansion reveals provenance, and review actions append events in an isolated workspace. Readers can therefore inspect what an answer claims, which evidence supports it and why its status changes.

3 Support, Locality and Transitive Exposure

3.1 Support Through Admissible Witnesses

A witness is a complete set of inputs that jointly justify a claim. A claim remains supported when at least one admissible witness succeeds. For its finite witness family $\mathcal{W}(c)$,

$$\mathrm{Supported}(c,t;q) = \bigvee_{W\in\mathcal{W}(c)} \left[\psi_c(W,t;q) \wedge \bigwedge_{u\in W} \mathrm{Valid}(u,t;q) \right]. \tag{5}$$

At knowledge time t for query q, Valid checks input availability, subject and time scope. Prerequisite claims must themselves remain supported. The predicate ψ_c checks the asserted interval, selected versions, quantity, normalized unit and numerical operation. Agreement uses $|v-\widehat{v}| \le \max(10^{-6}, 0.01|\widehat{v}|)$. A correction can therefore invalidate support while the evidence remains ~~availab~~le. Historical

node budget B and recorded times T,

$$H_{q,t} = \mathrm{View}_B(G_t, q), \quad p = \mathrm{Layout}\left(\bigcup_{u\in T} H_{q,u}; m\right), \quad I_{q,t} = \mathrm{Render}(H_{q,t}, p). \tag{4}$$

T is the finite set of displayed knowledge times; B bounds the number of rendered nodes. H is the bounded expansion, p stores coordinates and edge routes, and I is the drawing. Mode m selects deterministic Graphviz `dot` or seeded spring positioning. The spring layout uses an undirected projection for placement only. Analysis and arrows retain the directed, typed relationships. Time comparisons share coordinates, while records not yet ingested remain hidden. Layout changes presentation only. Distances represent neither semantic similarity nor neural state.

The implementation follows declared inputs within the selected database scope. Ingestion and assessment times control visibility, while event intervals specify when measurements or assertions apply. Original `DEPENDS_ON` citations remain intact. Derived paths trace newer versions by following supersession edges in reverse. Availability, support and display membership remain separate. Of 432 assessment groups sharing a record and time, 245 contain conflicting states (155 claim and 90 explanation groups). The runner inserts and assesses same-time revisions sequentially, exposing intermediate snapshots. Its recorded ingestion/version/ID order can reproduce that execution; arbitrary trigger-ID ordering cannot define an order-independent final snapshot. A general solution is to ingest a complete same-time batch before assessing dependents, requiring a batch boundary or watermark in a live stream. The exported view preserves the intermediate states and trigger IDs. Display membership follows the saved explanation version; grounding and recall score those actual displays against the event log, independently of the ambiguous assessment sidecar.

The node budget favors complete alternative support routes. Omissions or incomplete groups trigger a *partial view* warning and expansion control. Identity hubs, display records, assessments and reverse `SUPPORTS` links remain accessible in details. Structured views place jointly required features in labelled AND enclosures. Each row maps to a stored ID, and dashed bundles identify the dependency edges they summarize. Different groups provide alternative support routes.

Green, blue and grey identify claims, features and observations. Outlines and text show revisions and support status. Arrows retain their stored direction and labels or identify grouped edges. Figures 2 and 3 share typed JSON views, ID maps and rendering with the Streamlit dashboard. Its *Classic network* mode shows individual nodes (Figure 4). Controls select case, time, focus, route and record. Details expose full statements, exact values, units, intervals, versions, source IDs and assessments. Highlighting changes appearance only, expansion reveals provenance, and review actions append events in an isolated workspace. Readers can therefore inspect what an answer claims, which evidence supports it and why its status changes.

and current assertions are checked separately. The extension's inequalities use exact floating-point comparisons.

The original model uses local AND/OR rules and two-operand differences. Generated numerical claims require all declared parents and admissible direct numerical evidence. Comparisons cite both operands. The extension combines explicit AND groups and OR alternatives without cycles. Metadata preserves these witness groups, while source IDs store their combined inputs for traversal. Edges alone therefore cannot show which inputs are jointly required. Claims of missing evidence require a bounded set of possible producers and explicit availability tracking. Absence from the graph alone is insufficient.

As in provenance and truth maintenance [3,1], an alternative witness can preserve support when another fails. Our alternatives use disjoint recording windows, providing distinct measurement sources without assuming statistical independence.

3.2 Locality of a Revision

A correction identifies claims to recheck, but some may remain supported. Let U_δ contain the revised inputs, their earlier citable versions and changed records of evidence availability. Dependency edges point from claims to their inputs. Traversing these edges in reverse identifies potentially affected claims:

$$R_\delta = U_\delta \cup \mathrm{RevReach}(U_\delta), \qquad B_\delta = \{c : S_{t^-}(c) \neq S_{t^+}(c)\}, \tag{6}$$

Here $S_t(c)$ is the support function in Equation 5. R_δ contains the inputs and dependents to reconsider. B_δ contains only claims whose support actually changes.

Proposition 1 (Conditional locality). *For a finite acyclic dependency model, $B_\delta \subseteq R_\delta$ when every mutable input to each support predicate is explicitly represented and only U_δ changes between the two snapshots.*

Proof sketch. Consider nodes in order from inputs to dependents. A node outside R_δ has no changed input or prerequisite inside R_δ. Its numerical inputs, scope and availability therefore remain unchanged. Induction gives unchanged prerequisite support and hence unchanged support for the node itself. Including earlier versions preserves this argument for old citations, while availability records cover changes to missing-evidence claims. □

The result requires all relevant changes to be recorded. Untracked expiry, new support routes or external rule changes can violate this assumption. Our fixed-target queries and controlled witnesses represent changes through version or availability events. A benign correction or surviving alternative can leave a reachable claim supported, making B_δ smaller than R_δ.

With indexed adjacency lists, traversal costs $O(|V_{R_\delta}| + |E_{R_\delta}|)$. This excludes version resolution, support evaluation and database access. Our structural replays also evaluate the full small program, and Neo4j path queries may enumerate multiple paths. The bound describes graph traversal rather than total implementation runtime.

3 Support, Locality and Transitive Exposure

The following propositions specialize standard TMS/IVM reasoning to our declared DAG [6,11,4]; they are not new theoretical results. Direct-only checking is an ablation of transitive maintenance.

3.1 Support Through Admissible Witnesses

A witness is a complete set of inputs that jointly justify a claim. A claim remains supported when at least one admissible witness succeeds. For its finite witness family $\mathcal{W}(c)$,

$$\mathrm{Supported}(c, t; q) = \bigvee_{W \in \mathcal{W}(c)} \left[\psi_c(W, t; q) \wedge \bigwedge_{u \in W} \mathrm{Valid}(u, t; q) \right]. \tag{5}$$

At knowledge time t for query q, Valid checks input availability, subject and time scope. Prerequisite claims must themselves remain supported. The predicate ψ_c checks the asserted interval, selected versions, quantity, normalized unit and numerical operation. Agreement uses $|v - \widehat{v}| \leq \max(10^{-6}, 0.01|\widehat{v}|)$, where v is asserted and $\widehat{v}$ is computed from the selected evidence. A correction can therefore invalidate support while the evidence remains available. Historical and current assertions are checked separately. The extension's inequalities use exact floating-point comparisons.

The original model uses local AND/OR rules and two-operand differences. Generated numerical claims require all declared parents and admissible direct numerical evidence. Comparisons cite both operands. The extension combines explicit AND groups and OR alternatives without cycles. Metadata preserves these witness groups, while source IDs store their combined inputs for traversal. Edges alone therefore cannot show which inputs are jointly required. Claims of missing evidence require a bounded set of possible producers and explicit availability tracking. Absence from the graph alone is insufficient.

As in provenance and truth maintenance [10,6], an alternative witness can preserve support when another fails. Our alternatives use disjoint recording windows, providing distinct measurement sources without assuming statistical independence. Neighboring windows may be correlated; "alternative" means a distinct admissible route for the disjunctive query, not independent confirmation about the target alone.

3.2 Locality of a Revision

A correction identifies claims to recheck, but some may remain supported. Let U_δ contain the revised inputs, their earlier citable versions and changed records of evidence availability. Dependency edges point from claims to their inputs. Traversing these edges in reverse identifies potentially affected claims. The Neo4j adapter follows incoming `DEPENDS_ON` paths from revised inputs and their earlier

3.3 When Direct Maintenance Suffices

Direct maintenance checks claims immediately linked to changed inputs. It can miss claims that depend on those inputs through intermediate claims. Let A_δ contain the previously admitted claims that require a state change, as determined by an independent reference evaluation. Let D_δ contain the claims scheduled for direct checking. For $|A_\delta| > 0$, transitive exposure measures the fraction left unchecked:

$$\xi_\delta = \frac{|A_\delta \setminus D_\delta|}{|A_\delta|}. \tag{7}$$

We report NA when no change is required. The primary measure concerns withdrawal of initially grounded claims. Restoration is recorded separately. The event log and task specification determine A_δ independently of graph paths and acceptance flags.

Proposition 2 (Direct/full equivalence and misses). *Assume initially correct states, complete re-evaluation of D_δ, dependency semantics consistent with the reference evaluation, and no other process revisiting excluded claims. Before fresh generation, direct maintenance misses the fraction ξ_δ of required changes. Direct and full maintenance agree if every claim requiring a change is directly scheduled. This condition holds when every affected claim has a direct dependency edge to each changed input affecting its support and all other predicates remain unchanged.*

Proof sketch. Re-evaluation updates every required member of D_δ. Excluded claims retain their original states, so the missed set is exactly $A_\delta \setminus D_\delta$. When all required changes are scheduled, both methods evaluate the same support rules against the same evidence and produce the same states. □

Suppose both tests initially hold. Let c_1 state that test 1 holds, and c_2 state that tests 1 and 2 hold, citing c_1 and test 2. Changing test 1 to false invalidates both claims. Direct checking reaches only c_1, giving $\xi_\delta = 1/2$. Adding a direct citation from c_2 to test 1 brings both claims into the schedule and reduces exposure to zero. Alternatively, a surviving witness may preserve a downstream claim despite the correction. Revision therefore depends on which support is lost, as well as which claims are reachable.

4 Evaluation Design

4.1 Three-Source Evaluation

The initial study uses synthetic physiological waveforms, WESAD laboratory recordings ~~[10,9]~~, and PPG-DaLiA daily-activity recordings ~~[8,7]~~. Each source contributes ten held-out subjects with two episodes each. The real datasets also provide five separate development subjects, with splits fixed using PCG64 seed 20260928. Generation and validation use recorded features without physiological or activity labels.

versions, with distinct records and knowledge-time filters; the relational adapter uses recursive SQL:

$$R_\delta = U_\delta \cup \mathrm{RevReach}(U_\delta), \qquad B_\delta = \{c : S_{t^-}(c) \neq S_{t^+}(c)\}, \tag{6}$$

Here $S_t(c)$ is the support function in Equation 5. R_δ contains the inputs and dependents to reconsider. B_δ contains only claims whose support actually changes.

Proposition 1 (Conditional locality). *For a finite acyclic dependency model, $B_\delta \subseteq R_\delta$ when every mutable input to each support predicate is explicitly represented and only U_δ changes between the two snapshots.*

Proof sketch. Consider nodes in order from inputs to dependents. A node outside R_δ has no changed input or prerequisite inside R_δ. Its numerical inputs, scope and availability therefore remain unchanged. Induction gives unchanged prerequisite support and hence unchanged support for the node itself. Including earlier versions preserves this argument for old citations, while availability records cover changes to missing-evidence claims. □

The result requires all relevant changes to be recorded. Untracked expiry, new support routes or external rule changes can violate this assumption. Our fixed-target queries and controlled witnesses represent changes through version or availability events. A benign correction or surviving alternative can leave a reachable claim supported, making B_δ smaller than R_δ.

With indexed adjacency lists, traversal costs $O(|V_{R_\delta}| + |E_{R_\delta}|)$. This excludes version resolution, support evaluation and database access. Our structural replays also evaluate the full small program, and Neo4j path queries may enumerate multiple paths. The bound describes graph traversal rather than total implementation runtime.

3.3 When Direct Maintenance Suffices

Direct maintenance checks claims immediately linked to changed inputs. It can miss claims that depend on those inputs through intermediate claims. Let A_δ contain the previously admitted claims that require a state change, as determined by an independent reference evaluation. Let D_δ contain the claims scheduled for direct checking. For $|A_\delta| > 0$, transitive exposure measures the fraction left unchecked:

$$\xi_\delta = \frac{|A_\delta \setminus D_\delta|}{|A_\delta|}. \tag{7}$$

We report NA when no change is required. The primary measure concerns withdrawal of initially grounded claims. Restoration is recorded separately. The event log and task specification determine A_δ independently of graph paths and acceptance flags. Because A_δ already encodes required changes, ξ_δ is a retrospective accounting identity. Agreement with observed misses checks implementation consistency, not an independent predictive hypothesis.

Each episode compares a 30-second target window with the preceding 120 seconds. Four replay variants cover clean or benign updates, channel faults, delayed evidence and numerical corrections. Evaluation occurs 1, 31 and 91 seconds after the target window ends. Four question families cover the target EDA median, its difference from the preceding window, the fraction of missing EDA measurements and acceleration variability. Required facts are specified before generation.

Five methods produce 3,600 initial cases. B0 displays unchecked outputs. B1 checks direct dependencies in indexed records, while B2 uses Neo4j. M1 follows both direct and indirect dependencies in Neo4j. B3 applies the same support rules in indexed SQLite using recursive SQL queries. All checked methods validate new answers identically and allow at most one repair.

The initial study uses a fixed Qwen3-8B BF16 revision [5], allowing up to three claims and 1,536 output tokens. Both studies run inference on an RTX 5090 with vLLM structured decoding [12], with CUDA execution verified. Database operations and statistics run on the host CPU using Neo4j Community 5.26 and indexed SQLite. Three B0 transport failures are retained as empty outputs.

4.2 Independent Semantic Evaluation

We evaluate whether generated statements accurately describe the sensor data and provide the information requested. The facts concern measurable properties, including median skin conductance, changes between windows, missing measurements and acceleration variability.

For illustration, suppose a question asks for the target EDA median and its change from the preceding window. If the medians are 2.4 μS and 2.0 μS, a complete answer reports 2.4 μS and an increase of 0.4 μS. Reporting only the target median is accurate but covers half the requested information. Reporting only the preceding median gives correct background information without answering either requested fact.

Reference values are computed independently from saved sensor events and task specifications. The evaluator does not use the system's support flags or an LLM judge. Matching checks the dataset, subject, session, stated interval, quantity, operation, normalized value and unit, evidence version, and whether the statement concerns historical or current evidence. Citation and witness checks are separate because a correct number can still cite the wrong source.

For generated checkable claims $C(q,t)$ and answerable required facts $R(q,t)$, grounding precision measures factual accuracy and required-fact recall measures coverage:

$$\begin{aligned} P_{\text{grounded}} &= \frac{\sum_{c \in C} \mathbf{1}[\text{source-grounded } c]}{|C|}, \\ R_{\text{required}} &= \frac{|\{r \in R : \text{matched by a grounded } c\}|}{|R|}. \end{aligned} \tag{8}$$

The target-only answer above therefore has 100% precision and 50% recall. Each required fact counts once, so repetition cannot improve recall. The optional

Proposition 2 (Direct/full equivalence and misses). *Assume initially correct states, complete re-evaluation of D_δ, dependency semantics consistent with the reference evaluation, and no other process revisiting excluded claims. Before fresh generation, direct maintenance misses the fraction ξ_δ of required changes. Direct and full maintenance agree if every claim requiring a change is directly scheduled. This condition holds when every affected claim has a direct dependency edge to each changed input affecting its support and all other predicates remain unchanged.*

Proof sketch. Re-evaluation updates every required member of D_δ. Excluded claims retain their original states, so the missed set is exactly $A_\delta \setminus D_\delta$. When all required changes are scheduled, both methods evaluate the same support rules against the same evidence and produce the same states. □

Suppose both tests initially hold. Let c_1 state that test 1 holds, and c_2 state that tests 1 and 2 hold, citing c_1 and test 2. Changing test 1 to false invalidates both claims. Direct checking reaches only c_1, giving $\xi_\delta = 1/2$. Adding a direct citation from c_2 to test 1 brings both claims into the schedule and reduces exposure to zero. Alternatively, a surviving witness may preserve a downstream claim despite the correction. Revision therefore depends on which support is lost, as well as which claims are reachable.

4 Evaluation Design

4.1 Three-Source Evaluation

The initial study uses synthetic physiological waveforms, WESAD laboratory recordings [21,20], and PPG-DaLiA daily-activity recordings [19,18]. Each source contributes ten held-out subjects with two episodes each. The real datasets also provide five separate development subjects, with splits fixed using PCG64 seed 20260928. Generation and validation use recorded features without physiological or activity labels.

Each episode compares a 30-second target window with the preceding 120 seconds. Four replay variants cover clean or benign updates, channel faults, delayed evidence and numerical corrections. Evaluation occurs 1, 31 and 91 seconds after the target window ends. All four question families request the target EDA median. Current-evidence and conflict-timing questions also request its difference from the preceding window; sufficiency requests missing fraction, and revision-impact requests acceleration variability. Required facts are specified before generation. Each episode is assigned one question family, retained across four variants and three checkpoints: 240 answers per source, 60 per family. Each answer requests two slots; 420 of 480 slots are answerable (100 current-evidence, 100 conflict-timing, 110 sufficiency and 110 revision-impact slots). The 260/290/290 denominators below instead concern the 160 paired revision transitions and change when delayed evidence arrives.

Five methods produce 3,600 initial cases. B0 displays unchecked outputs. B1 checks direct dependencies in indexed records, while B2 uses Neo4j. M1 follows

harmonic summary $S = 2P_{\text{grounded}}R_{\text{required}}/(P_{\text{grounded}} + R_{\text{required}})$ combines these measures. It is not conventional set F1 because generated claims and required facts have different counting bases. For answerable questions, empty output has precision NA and zero recall and summary score. Fully unanswerable questions have recall and summary NA, with missing-evidence statements checked separately. Partially answerable questions are scored against the facts that can be established. We score raw proposals, proposals after optional repair, accepted claims and displayed answers separately. A limited prose audit checks literal discrepancies and interpretive wording. To evaluate revision, we also score the same prior answer before new evidence arrives, immediately afterward and after maintenance. These checkpoints distinguish removing unsupported content from supplying a complete replacement, without implying a continuous recovery curve.

4.3 Controlled Structural Extension

We test when indirect dependencies and alternative evidence affect revision. Before generation, we select the first five held-out subjects per source in natural order and both episodes, reusing 30 episodes from the initial study. Four requested dependency depths (1–4) and two support regimes produce 240 cases.

Each case asks whether at least one eligible sensor window passes four checks on EDA median, EDA interquartile range, acceleration standard deviation and EDA missing fraction. Thresholds are fixed from the clean episode so all checks initially pass. Cases provide either one supporting window or an alternative from the disjoint preceding window. Depth one links the measurements directly to the answer. Greater depths introduce intermediate claims about individual tests, groups of tests and complete windows.

Exact programs implement these structures. Separately, the LLM proposes up to 16 claims and their supporting inputs, with a 4,096-token output limit and at most one repair. We evaluate the links actually generated, retaining missing answers, incomplete alternatives and differences between requested and achieved depth. Two development pilots use 40 separate calls before the main protocol is fixed. The second clarifies readable statements, grouping and references to earlier claims. Neither pilots nor the compiler insert missing model links.

All methods receive the same admitted candidate, evidence updates and support rules, with no generation after correction. B0 leaves the candidate unchanged. Four separate updates invalidate the first test in every route, invalidate only the primary route, make a harmless numerical correction or change an unused cardiac feature. Neo4j and relational implementations process identical inputs. Independent truth-table checks test whether the proposed witnesses justify the claims and capture the required alternatives, allowing different valid derivations. A separate evaluation of the event log determines which claims must change.

4.4 Graph Measurement and Uncertainty

Read-only Neo4j exports preserve nodes, relations, assessment times, Cypher and element IDs. Counts match the original ledger and sampled stored paths.

both direct and indirect dependencies in Neo4j. B3 applies the same support rules in indexed SQLite using recursive SQL queries. All checked methods validate new answers identically and allow at most one repair.

The initial study uses a fixed Qwen3-8B BF16 revision [15], allowing up to three claims and 1,536 output tokens. Both studies run inference on an RTX 5090 with vLLM structured decoding [23], with CUDA execution verified. Database operations and statistics run on the host CPU using Neo4j Community 5.26 and indexed SQLite. Three B0 transport failures are retained as empty outputs.

4.2 Independent Semantic Evaluation

We evaluate whether generated statements accurately describe the sensor data and provide the information requested. The facts concern measurable properties, including median skin conductance, changes between windows, missing measurements and acceleration variability.

For illustration, suppose a question asks for the target EDA median and its change from the preceding window. If the medians are 2.4 μS and 2.0 μS, a complete answer reports 2.4 μS and an increase of 0.4 μS. Reporting only the target median is accurate but covers half the requested information. Reporting only the preceding median gives correct background information without answering either requested fact.

Reference values are computed independently from saved sensor events and task specifications. The evaluator does not use the system's support flags or an LLM judge. Admission and scoring nevertheless share the numerical quantities, intervals, units and tolerance, so perfect post-filter grounding is expected under that contract and is not independent evidence of general faithfulness. Matching checks the dataset, subject, session, stated interval, quantity, operation, normalized value and unit, evidence version, and whether the statement concerns historical or current evidence. Citation and witness checks are separate because a correct number can still cite the wrong source.

For generated checkable claims $C(q,t)$ and answerable required facts $R(q,t)$, grounding precision measures factual accuracy and required-fact recall measures coverage:

$$P_{\text{grounded}} = \frac{\sum_{c \in C} \mathbf{1}[\text{source-grounded } c]}{|C|}, \qquad R_{\text{required}} = \frac{|\{r \in R : \text{matched by a grounded } c\}|}{|R|}. \tag{8}$$

The target-only answer above therefore has 100% precision and 50% recall. Each required fact counts once, so repetition cannot improve recall. The optional harmonic summary $S = 2P_{\text{grounded}}R_{\text{required}}/(P_{\text{grounded}} + R_{\text{required}})$ combines these measures. It is not conventional set F1 because generated claims and required facts have different counting bases. For answerable questions, empty output has precision NA and zero recall and summary score. Fully unanswerable questions have recall and summary NA, with missing-evidence statements checked separately. Partially answerable questions are scored against the facts that can

Separate measurements of the full graph, active graph and query core cover type mixing, degree, depth, direct citations, witness counts, reachability and actual changes. Dependency degree excludes SUPPORTS and provenance mirrors. Meaningful zero categories remain, and we impose no power-law or embedding interpretation.

We first aggregate snapshots and variants within episodes, then episodes within subjects. Pointwise 95% intervals use 2,000 subject-cluster bootstrap resamples with seed 20260929. Historical versions and replay copies never count as independent participants. Detailed distributions and operational logs remain repository data. This paper retains the definitions and primary denominators needed to interpret the results.

5 Results

5.1 A Compact, Traceable View of Each Explanation

The graph database preserves the evidence and revision history behind generated answers, while a smaller query core brings the relevant claims and inputs together for inspection. Across 480 retained Neo4j scopes, M1 stores 47,038 nodes and 128,789 relations. These records include measurement versions, source links and replay histories. Exported counts match the original ledger, and checks of 1850 representative paths confirm that the analysis preserves the stored connections.

For an individual answer, the query core averages only 2.8 to 4.1 nodes, representing 1.5% to 2.2% of visible storage. This gives the reviewer a focused starting point for tracing what an answer claims and which inputs it uses. Linked metadata retains access to the underlying observation windows, while expanded views reveal earlier versions and subsequent corrections. The database therefore supports both a compact explanation view and a detailed evidence trail. We analyze claim dependencies separately from administrative links and aggregate statistics within subjects, so repeated replays do not become independent observations.

Figure 2 illustrates the practical value of this representation. A corrected measurement causes two numerical claims to be withdrawn, while their original evidence references remain available to explain the decision.

be established. We score raw proposals, proposals after optional repair, accepted claims and displayed answers separately. A limited prose audit checks literal discrepancies and interpretive wording. To evaluate revision, we also score the same prior answer before new evidence arrives, immediately afterward and after maintenance. These checkpoints distinguish removing unsupported content from supplying a complete replacement, without implying a continuous recovery curve.

4.3 Controlled Structural Extension

We test when indirect dependencies and alternative evidence affect revision. Before generation, we select the first five held-out subjects per source in natural order and both episodes, reusing 30 episodes from 15 dataset-scoped subjects. Four requested dependency depths (1–4) and two support regimes produce 240 cases.

Each case asks whether at least one eligible sensor window passes four checks on EDA median, EDA interquartile range, acceleration standard deviation and EDA missing fraction. Thresholds are fixed from the clean episode so all checks initially pass. Cases provide either one supporting window or an alternative from the disjoint preceding window. Depth one links the measurements directly to the answer. Greater depths introduce intermediate claims about individual tests, groups of tests and complete windows.

Exact programs implement these structures. Separately, the LLM proposes up to 16 claims and their supporting inputs, with a 4,096-token output limit and at most one repair. We evaluate the links actually generated, retaining missing answers, incomplete alternatives and differences between requested and achieved depth. Two development pilots use 40 separate calls before the main protocol is fixed. The second clarifies readable statements, grouping and references to earlier claims. Neither pilots nor the compiler insert missing model links.

The primary analysis uses the existing corrected-admission replication, which accepts primitive test names omitted from the original admission table. Candidate text and declared dependencies are unchanged; exact fixtures are unaffected. All methods receive the same admitted candidate, evidence updates and support rules, with no generation after correction. B0 leaves the candidate unchanged. Four separate updates invalidate the first test in every route, invalidate only the primary route, make a harmless numerical correction or change an unused cardiac feature. Neo4j and relational implementations process identical inputs. Independent truth-table checks test whether the proposed witnesses justify the claims and capture the required alternatives, allowing different valid derivations. A separate evaluation of the event log determines which claims must change.

4.4 Graph Measurement and Uncertainty

Read-only Neo4j exports preserve nodes, relations, assessment times, Cypher and element IDs. Counts match the original ledger and sampled stored paths. Separate measurements of the full graph, active graph and query core cover type mixing, degree, depth, direct citations, witness counts, reachability and

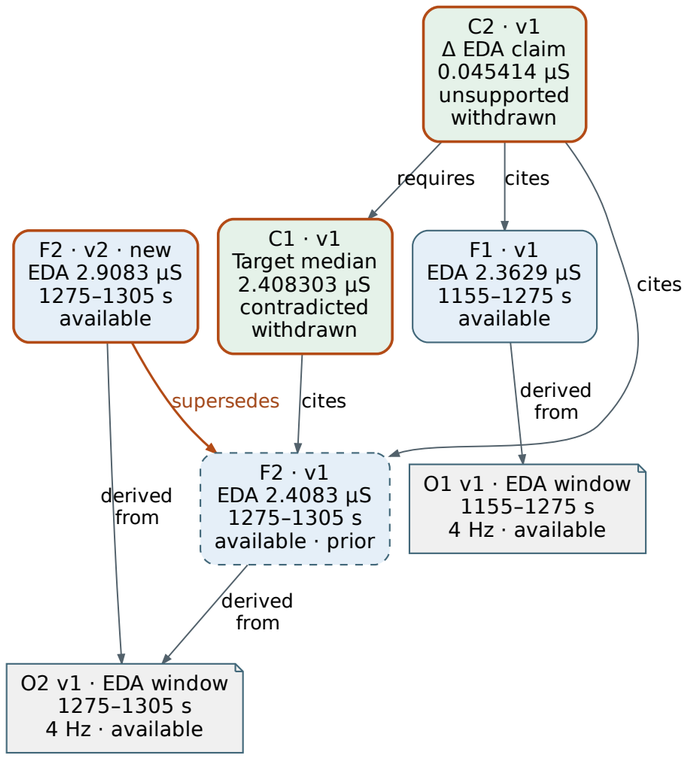


Fig. 2. Tracing a correction through an actual WESAD explanation stored in Neo4j. For subject S11, episode 0, the target EDA median changes from 2.4083 to 2.9083 μS. The old median claim C1 and comparison C2 are withdrawn, while their original citations remain visible. Both claims directly cite the corrected input, so direct checking reaches both. The view is shown at knowledge time 1336.01 s and selected by the median-size rule from 20 eligible cases. Green nodes represent claims, blue nodes features, and folded corners observation windows. *Cites/requires* denotes `DEPENDS_ON`, *derived from* denotes `DERIVED_FROM`, and *supersedes* connects the new version to the old. A measurement can remain available without supporting an earlier claim. Administrative objects and reverse support links remain accessible in details. Displayed values are rounded.

actual changes. Dependency degree excludes SUPPORTS and provenance mirrors. Meaningful zero categories remain, and we impose no power-law or embedding interpretation.

We first aggregate snapshots and variants within episodes, then episodes within subjects. Pointwise 95% intervals use 2,000 subject-cluster bootstrap resamples with seed 20260929. Historical versions and replay copies never count as independent participants. Invariant scores and deterministic fixture outcomes are reported without zero-width intervals. Detailed distributions and operational logs remain repository data. This paper retains the definitions and primary denominators needed to interpret the results. Original code and frozen artifacts are available at `https://github.com/mantzaris/EAI-BDCC2026`; the author-identified final version uses this public link. The added baseline script and results accompany the source archive and have not yet been published at that URL.

5 Results

5.1 A Compact, Traceable View of Each Explanation

The graph database preserves the evidence and revision history behind generated answers, while a smaller query core brings the relevant claims and inputs together for inspection. Across 480 retained Neo4j scopes, M1 stores 47,038 nodes and 128,789 relations. These records include measurement versions, source links and replay histories. Exported counts match the original ledger, and checks of 1850 representative paths confirm that the analysis preserves the stored connections.

For an individual answer, the query core averages only 2.8 to 4.1 nodes, representing 1.5% to 2.2% of visible storage. This provides a compact view for tracing what an answer claims and which inputs it uses. Linked metadata retains access to the underlying observation windows, while expanded views reveal earlier versions and subsequent corrections. The database therefore supports both a compact explanation view and a detailed evidence trail. We analyze claim dependencies separately from administrative links and aggregate statistics within subjects, so repeated replays do not become independent observations.

Figure 2 illustrates this representation. A corrected measurement causes two numerical claims to be withdrawn, while their original evidence references remain available to explain the decision.

Only 18 of 1201 displayed M1 claims cite another claim, leaving 98.5% with no claim parents. Claim-to-claim chains never exceed one edge. These links do not necessarily require transitive updates. In Figure 2, the comparison also cites the corrected feature directly. Across 360 revision batches, 210 require withdrawal and 0 have nonzero exposure. The locality audit finds 0 violations. Direct and full maintenance agree because direct checking already reaches every required change. These prompts are task-guided and schema-constrained; no frequency for unconstrained generation is established.

The ontology in practice. Figure 4 maps the admitted PPG-DaLiA program to the ontology in Equations 1-2. Each node is a stored record. For example, A1 is

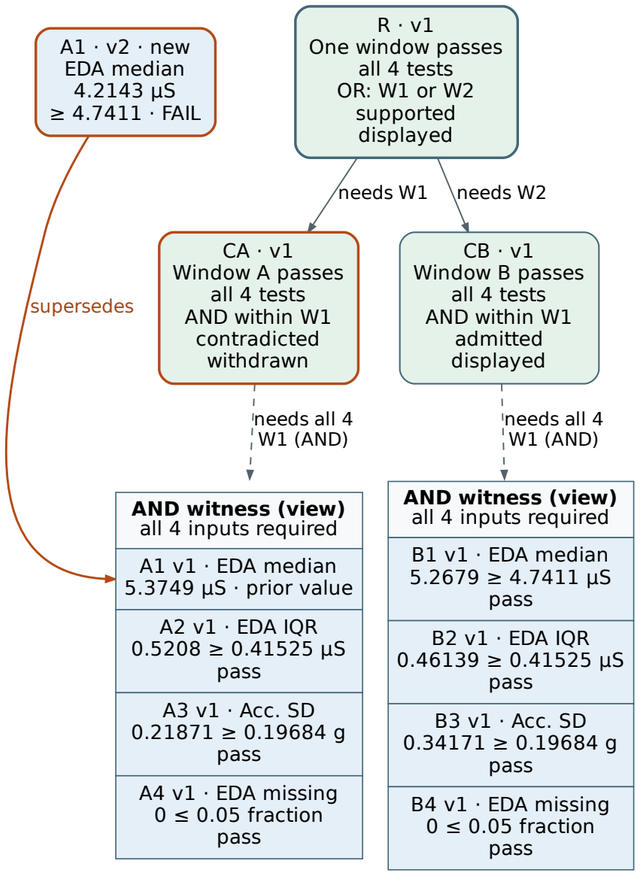


Fig. 3. Making generated answers checkable and selectively revisable as evidence changes. Stored in Neo4j, this network connects model-generated claims to the measurements and alternative routes that justify them. Correcting A1 invalidates claim CA, while window B still passes all four checks and supports the final answer R that at least one window qualifies. The system therefore withdraws the unsupported statement and preserves the justified conclusion. The same graph exposes the evidence behind both decisions for human review, connecting automated reliability checks with an inspectable explanation. Each route requires all four blue feature inputs, while the answer requires only one valid route. Dashed arrows group four stored dependency edges within each AND enclosure. Solid *needs* arrows connect dependent claims. This PPG-DaLiA S1 episode 0 example is shown at knowledge time 2331.01 s, with windows A and B covering 2270–2300 and 2150–2270 s. Requested and realized depth are both 2. The example is the smallest complete graph among 87 eligible programs, with ties resolved by ID.

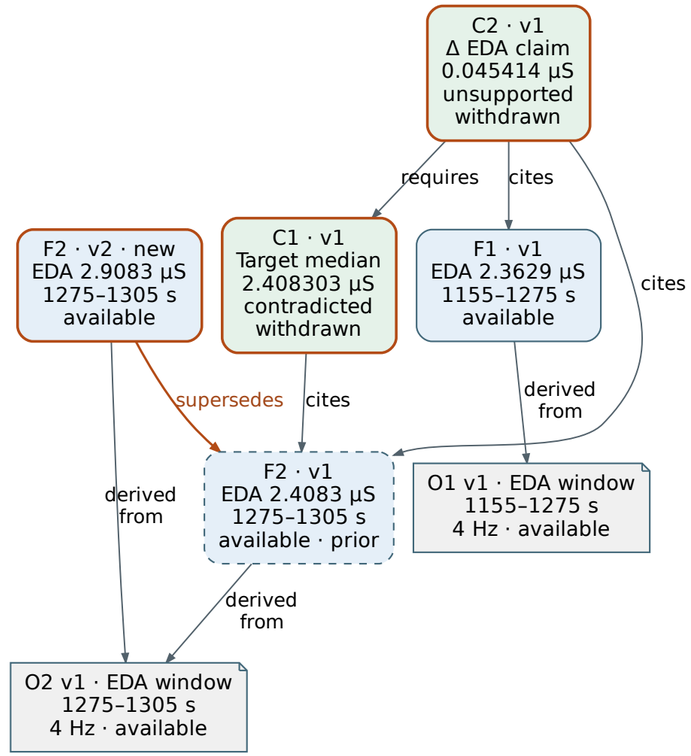


Fig. 2. Tracing a correction through an actual WESAD explanation stored in Neo4j. For subject S11, episode 0, the target EDA median changes from 2.4083 to 2.9083 μS. The old median claim C1 and comparison C2 are withdrawn, while their original citations remain visible. Both claims directly cite the corrected input, so direct checking reaches both. The view is shown at knowledge time 1336.01 s and selected by the median-size rule from 20 eligible cases. Green nodes represent claims, blue nodes features, and folded corners observation windows. *Cites/requires* denotes `DEPENDS_ON`, *derived from* denotes `DERIVED_FROM`, and *supersedes* connects the new version to the old. A measurement can remain available without supporting an earlier claim. Administrative objects and reverse support links remain accessible in details. Displayed values are rounded.

Red: removed/replaced. Blue: added/replacement. Page pairing is positional.

Only 18 of 1201 displayed M1 claims cite another claim, leaving 98.5% with no claim parents. Claim-to-claim chains never exceed one edge. These links do not necessarily require transitive updates. In Figure 2, the comparison also cites the corrected feature directly. Across 360 revision batches, 210 require withdrawal and 0 have nonzero exposure. The locality audit finds 0 violations. Direct and full maintenance agree because direct checking already reaches every required change.

The ontology in practice. Figure 4 maps the admitted PPG-DaLiA program to the ontology in Equations 1-2. Each node is a stored record. For example, A1 is a specific `FeatureVersion`. The view selects 14 records and 18 relations from 26 objects and 83 relations. The 11 starred records form the current answer core (Equation 3). CA remains linked to R after withdrawal, preserving the declared dependency for inspection. The earlier A1 version and two EDA windows show the evidence history. Motion-window provenance can also be expanded.

Each route requires four tests, and either route can support the answer. For this program, Equation 5 becomes

$$S(C_A, t) = b_{A1}(t) \wedge b_{A2}(t) \wedge b_{A3}(t) \wedge b_{A4}(t)$$
$$S(C_B, t) = b_{B1}(t) \wedge b_{B2}(t) \wedge b_{B3}(t) \wedge b_{B4}(t)$$
$$S(R, t) = S(C_A, t) \vee S(C_B, t)$$

Each b_{ri} checks the latest visible feature for PPG-DaLiA S1, session 1, with the specified A/B interval, quantity and unit and an available non-null value. It then applies the exact $\geq$ threshold for median, IQR and acceleration SD, or $\leq$ for missing fraction (Figure 3). A1 changes from 5.374856 μS, above 4.74108975 μS, to 4.214302 μS, below it. The measurement remains available, but b_{A1} becomes false. The witness lists therefore give $(S(C_A), S(C_B), S(R)) : (1, 1, 1) \to (0, 1, 1)$.

The theorem's dependency projection contains 13 nodes and 10 edges. Reachability excludes provenance, membership and support mirrors. Equation 6 gives $U_\delta = \{\mathrm{A1}_{v1}, \mathrm{A1}_{v2}\}$ and $R_\delta = U_\delta \cup \{C_A, R\}$, but $B_\delta = \{C_A\}$. These full computed sets are all visible here. The independent event/task oracle and recorded B2 schedule separately give $A_\delta = D_\delta = \{C_A\}$. Exposure in Equation 7 is $0/1 = 0$. Two dependent claims need re-evaluation, yet only CA changes support because another witness preserves the root. Direct and full maintenance agree.

5.2 Grounded Content and Useful Coverage

Emitted-claim counts before arrival, after arrival and after maintenance are Synthetic 211/211/138, WESAD 281/281/207, and PPG-DaLiA 271/271/191. Each source has 260/290/290 answerable required facts at these checkpoints. Precision excludes empty outputs, whereas recall includes their zero coverage. These pooled counts describe the sample. Reported percentages average episodes within subjects.

Checking produces fully grounded displays under the bounded numerical protocol, but it also removes useful information.

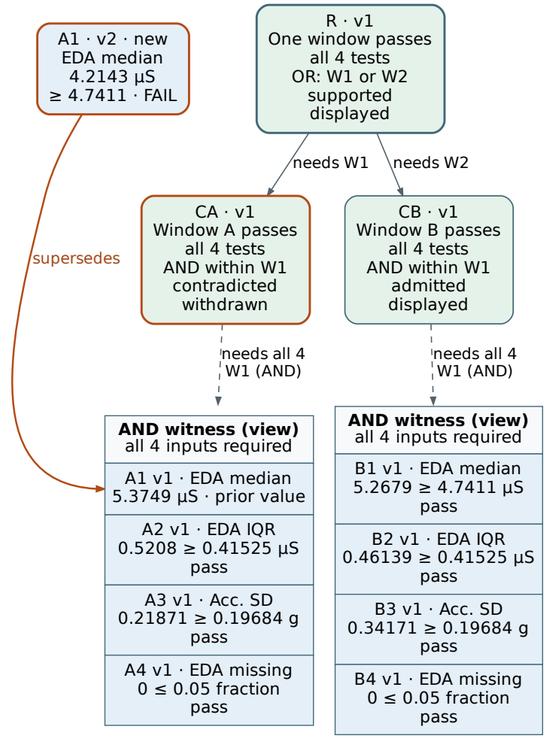


Fig. 3. Making generated answers checkable and selectively revisable as evidence changes. Stored in Neo4j, this network connects model-generated claims to the measurements and alternative routes that justify them. Correcting A1 invalidates claim CA, while window B still passes all four checks and supports the final answer R that at least one window qualifies. Each route requires all four blue feature inputs, while the answer requires only one valid route. Dashed arrows group four stored dependency edges within each AND enclosure. Solid *needs* arrows connect dependent claims. This PPG-DaLiA S1 episode 0 example is shown at knowledge time 2331.01 s, with windows A and B covering 2270–2300 and 2150–2270 s. Requested and realized depth are both 2. The example is the smallest complete graph among 87 originally eligible programs, with ties resolved by ID.

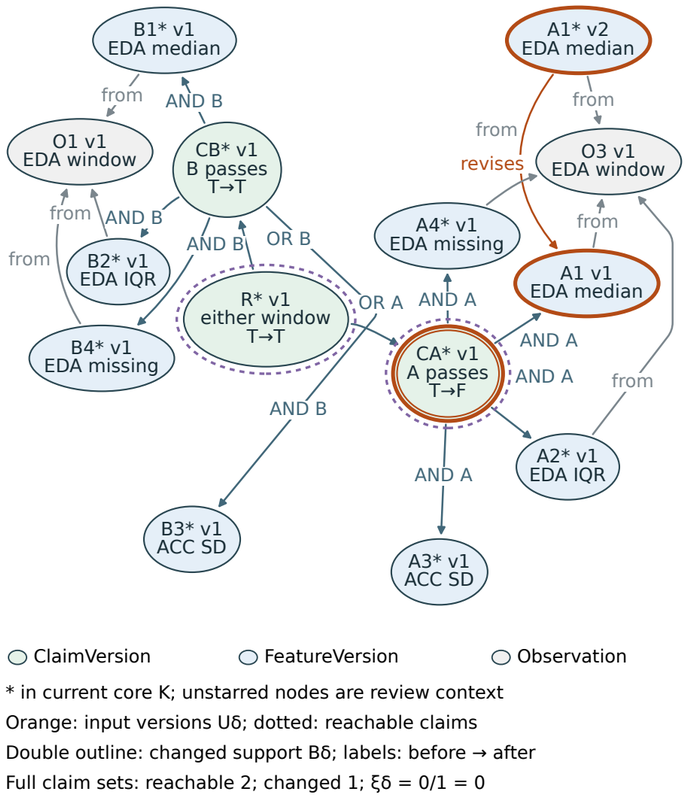


Fig. 4. Tracing how corrected evidence changes a generated answer. In this PPG-DaLiA S1 episode 0 example, CA loses support while CB keeps the final answer R valid. All 14 nodes are Neo4j records. AND labels mark inputs required together, while OR labels show alternative support. *From* links features to observations, and *revises* links new versions to old. Colors distinguish record types, outlines mark revision status, and stars identify the answer core. The view compares 2301.01 and 2331.01 s around the correction at 2331.0 s. Original citations remain intact, with further provenance available on expansion.

a specific `FeatureVersion`. The view selects 14 records and 18 relations from 26 objects and 83 relations. The 11 starred records form the current answer core (Equation 3). CA remains linked to R after withdrawal, preserving the declared dependency for inspection. The earlier A1 version and two EDA windows show the evidence history. Motion-window provenance can also be expanded.

Each route requires four tests, and either route can support the answer. For this program, Equation 5 becomes

$$S(C_A, t) = b_{A1}(t) \wedge b_{A2}(t) \wedge b_{A3}(t) \wedge b_{A4}(t)$$
$$S(C_B, t) = b_{B1}(t) \wedge b_{B2}(t) \wedge b_{B3}(t) \wedge b_{B4}(t)$$
$$S(R, t) = S(C_A, t) \vee S(C_B, t)$$

Each b_{ri} checks the latest visible feature for PPG-DaLiA S1, session 1, with the specified A/B interval, quantity and unit and an available non-null value. It then applies the exact $\geq$ threshold for median, IQR and acceleration SD, or $\leq$ for missing fraction (Figure 3). A1 changes from 5.374856 μS, above 4.74108975 μS, to 4.214302 μS, below it. The measurement remains available, but b_{A1} becomes false. The witness lists therefore give $(S(C_A), S(C_B), S(R)) : (1, 1, 1) \to (0, 1, 1)$.

The theorem's dependency projection contains 13 nodes and 10 edges. Reachability excludes provenance, membership and support mirrors. Equation 6 gives $U_\delta = \{\mathrm{A1}_{v1}, \mathrm{A1}_{v2}\}$ and $R_\delta = U_\delta \cup \{C_A, R\}$, but $B_\delta = \{C_A\}$. These full computed sets are all visible here. The independent event/task oracle and recorded B2 schedule separately give $A_\delta = D_\delta = \{C_A\}$. Exposure in Equation 7 is $0/1 = 0$. Two dependent claims need re-evaluation, yet only CA changes support because another witness preserves the root. Direct and full maintenance agree.

5.2 Grounded Content and Useful Coverage

Emitted-claim counts before arrival, after arrival and after maintenance are Synthetic 211/211/138, WESAD 281/281/207, and PPG-DaLiA 271/271/191. Each source has 260/290/290 answerable required facts at these checkpoints. Precision excludes empty outputs, whereas recall includes their zero coverage. These pooled counts describe the sample. Reported percentages average episodes within subjects.

Checking produces fully grounded displays under the bounded numerical protocol, but it also removes useful information. There are 20 empty displays among 240 answers per source, not 420 displays; 420 counts answerable fact slots. In these saved M1 cases, all 20 empty answers have no answerable slots. The general scoring rule still assigns zero recall to an empty answerable response.

Raw M1 required recall is 95.0%, 87.0% and 88.8% for synthetic, WESAD and PPG-DaLiA respectively. Displayed recall falls to 78.2%, 75.8% and 75.5% (Table 3). Thus the precision gain partly comes from rejecting correct, useful propositions. Figure 5 follows the same prior answers. Withdrawing stale assertions restores grounding among retained claims, but required facts can remain unanswered.

Table 3. Evidence checking raises M1 grounding from 83.4-87.3% in raw proposals to 100% in displayed claims across all three sources, while retaining ~~75.5-78.2%~~ of required facts. P measures whether statements are supported by the source data. R measures how completely they answer the question. Values are percentages ~~with subject means and 95% intervals.~~ Each row covers 240 answers from 20 episodes. Matched reports requirements met over answerable requirements, counting each requirement once per answer. Empty displays have zero recall on answerable questions and undefined precision. These semantic scores are separate from the original strict output-contract checks.

Source	State	P [95% CI]	R [95% CI]	Matched	Empty
Synthetic	B0 display	87.8 [84.5, 90.5]	95.5 [90.0, 99.5]	402/420	1
Synthetic	M1 raw	87.3 [83.4, 90.4]	95.0 [88.5, 100.0]	400/420	0
Synthetic	M1 display	100.0 ~~[100.0, 100.0]~~	78.2 [66.2, 90.0]	333/420	20
WESAD	B0 display	83.6 [77.2, 89.4]	86.5 [77.8, 95.0]	366/420	1
WESAD	M1 raw	83.4 [77.1, 89.2]	87.0 [78.2, 95.5]	368/420	0
WESAD	M1 display	100.0 ~~[100.0, 100.0]~~	75.8 [64.0, 87.5]	323/420	20
PPG-DaLiA	B0 display	84.5 [78.7, 89.6]	87.8 [79.2, 94.8]	371/420	1
PPG-DaLiA	M1 raw	84.7 [79.2, 89.6]	88.8 [80.7, 95.5]	375/420	0
PPG-DaLiA	M1 display	100.0 ~~[100.0, 100.0]~~	75.5 [65.0, 87.5]	322/420	20

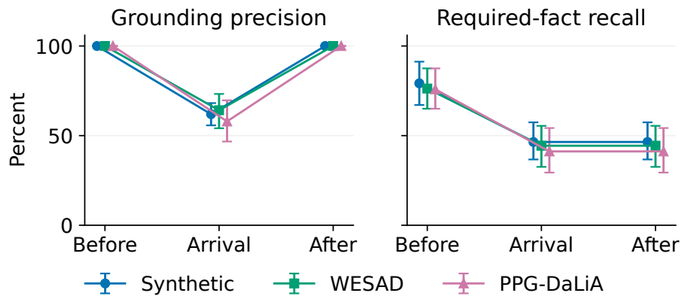


Fig. 5. Independent grounding precision and required recall for the same prior M1 answers at three discrete checkpoints, with subject-cluster 95% intervals. Each source has 160 transitions from 20 episodes and ten subjects across the four variants. No smooth recovery curve is inferred. Emitted claim and required-fact denominators appear in the text. Fresh-answer scores remain in Table 3.

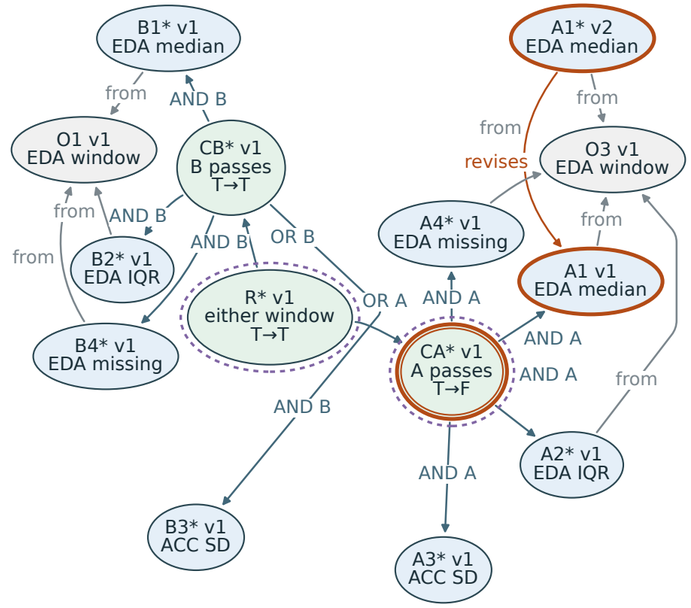


Fig. 4. Tracing how corrected evidence changes a generated answer. In this PPG-DaLiA S1 episode 0 example, CA loses support while CB keeps the final answer R valid. All 14 nodes are Neo4j records. AND labels mark inputs required together, while OR labels show alternative support. *From* links features to observations, and *revises* links new versions to old. Colors distinguish record types, outlines mark revision status, and stars identify the answer core. The view compares 2301.01 and 2331.01 s around the correction at 2331.0 s. Original citations remain intact, with further provenance available on expansion.

Raw M1 required recall is 95.0%, 87.0% and 88.8% for synthetic, WESAD and PPG-DaLiA respectively. Displayed recall falls to 78.2%, 75.8% and 75.5% (Table 3). Thus the precision gain partly comes from rejecting correct, useful propositions. Figure 5 follows the same prior answers. Withdrawing stale assertions restores grounding among retained claims, but required facts can remain unanswered.

Of 799 B0 claims rejected by the frozen contract, 499 are grounded under the new proposition protocol. This includes 333 correctly time-labelled background observations. Other disagreements arise when dependency or citation restrictions reject correct numerical content. No claim that passed the old contract becomes numerically ungrounded under the new scoring. A wrong target attribution still fails because matching uses the asserted interval and independently resolved version.

Queries about source windows and known versions agree with the independent log oracle on all 1201 feature witnesses. Queries about comparison operands agree on 18 eligible displayed comparisons. These checks recover admissible witnesses without requiring a unique gold reasoning graph. The separate prose audit and original auxiliary-verifier disagreements remain available. The bounded scores do not certify unrestricted interpretation.

5.3 Structural Variation Tests the Maintenance Condition

Table 4. Common-candidate structural replay. Counts aggregate four update conditions per case, separately for exact and generated programs. A required change is independently determined before scheduling. Correction intervals average episodes within subjects identified separately for each dataset. B1 matches B2 and B3 matches M1 in every cell. Missing generated roots are included in the depth/coverage analysis, not invented as initially displayed claims.

Program	Method	Required	Missed	Collateral	Corrected % [CI]
Exact	B0	1260	1260	0	0.0 [0.0, 0.0]
Exact	B2	1260	720	0	42.9 [42.9, 42.9]
Exact	M1	1260	0	0	100.0 [100.0, 100.0]
Generated	B0	1210	1210	0	0.0 [0.0, 0.0]
Generated	B2	1210	183	0	84.9 [84.0, 85.7]
Generated	M1	1210	0	0	100.0 [100.0, 100.0]

Across 240 cases and 9,600 replays, full maintenance corrects every required change without withdrawing unaffected claims. Direct maintenance misses 720 of 1260 required changes in exact programs and 183 of 1210 in generated programs. Transitive exposure predicts these misses in all 860 of 860 eligible groups (Figure 6). Neo4j and the relational control agree across all 1,920 matched groups.

Alternative support preserves all 120 exact and 96 generated two-route answers after the primary route fails (Figure 3). Single-route answers lose support.

Table 3. Evidence checking raises M1 grounding from 83.4-87.3% in raw proposals to 100% in displayed claims across all three sources, while required-fact recall falls from 87.0–95.0% to 75.5–78.2%. P measures whether statements are supported by the source data. R measures how completely they answer the question. Values are subject-mean percentages; 95% intervals are omitted for invariant results. Each row covers 240 answers from 20 episodes. Matched reports requirements met over answerable requirements, counting each requirement once per answer. Empty displays have zero recall on answerable questions and undefined precision. These semantic scores are separate from the original strict output-contract checks.

Source	State	P [95% CI]	R [95% CI]	Matched	Empty
Synthetic	B0 display	87.8 [84.5, 90.5]	95.5 [90.0, 99.5]	402/420	1
Synthetic	M1 raw	87.3 [83.4, 90.4]	95.0 [88.5, 100.0]	400/420	0
Synthetic	M1 display	100.0	78.2 [66.2, 90.0]	333/420	20
WESAD	B0 display	83.6 [77.2, 89.4]	86.5 [77.8, 95.0]	366/420	1
WESAD	M1 raw	83.4 [77.1, 89.2]	87.0 [78.2, 95.5]	368/420	0
WESAD	M1 display	100.0	75.8 [64.0, 87.5]	323/420	20
PPG-DaLiA	B0 display	84.5 [78.7, 89.6]	87.8 [79.2, 94.8]	371/420	1
PPG-DaLiA	M1 raw	84.7 [79.2, 89.6]	88.8 [80.7, 95.5]	375/420	0
PPG-DaLiA	M1 display	100.0	75.5 [65.0, 87.5]	322/420	20

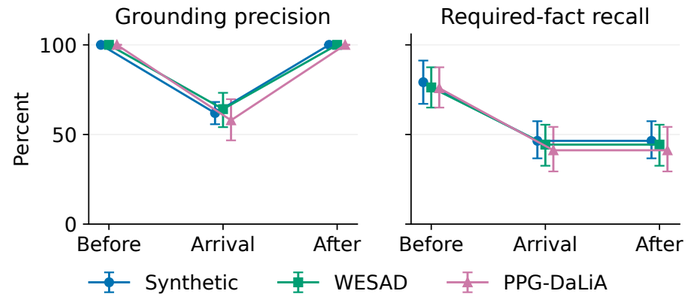


Fig. 5. Independent grounding precision and required recall for the same prior M1 answers at three discrete checkpoints, with subject-cluster 95% intervals. Each source has 160 transitions from 20 episodes and ten subjects across the four variants. No smooth recovery curve is inferred. Emitted claim and required-fact denominators appear in the text. Fresh-answer scores remain in Table 3.

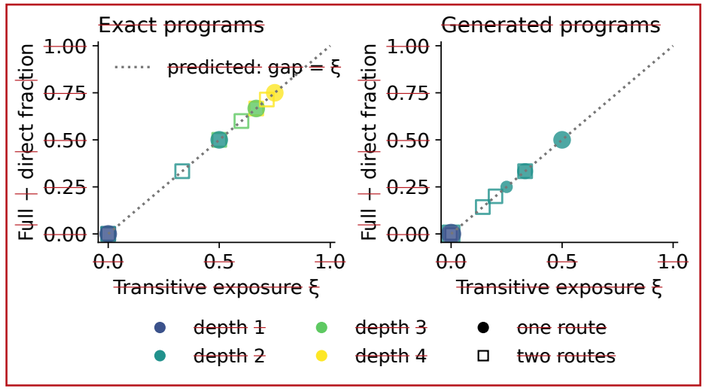


Fig. 6. Predicting when indirect dependencies matter for reliable updates. The horizontal axis shows the fraction of required changes missed by the direct checking schedule. The vertical axis shows the additional fraction corrected by full maintenance. Results follow the diagonal for both exact and GPU-generated programs, showing that transitive exposure predicts the benefit of following indirect dependencies. Points group batches with the same exposure, support regime and realized depth. Larger points represent more batches. These results use the original admission rules. The text reports a separate sensitivity analysis after correcting admission normalization.

Generated answers reach depth one in 74 cases and depth two in 135, while 31 cases yield no admitted answer. None reaches depth three or four, so exact programs provide the deeper tests. In 236 paired cases, identical evidence and record counts produce different exposure. What matters is which inputs an answer requires and whether another valid route survives.

A separate replication corrects an admission-table omission that rejected supplied test names. Model outputs and dependencies remain unchanged. Across 4,800 replays, direct maintenance misses 208 of 1294 required changes, while full maintenance again misses none. This confirms the maintenance result under the corrected admission rules. Original scores remain unchanged, and independent truth-table checks assess whether the declared support is sufficient and complete.

6 Discussion and Conclusion

This study demonstrates how a graph database can support reliable, inspectable generative AI. The method connects generated statements to their measurements, supporting claims and evidence versions in Neo4j. These connections guide revision when evidence changes and produce network diagrams that explain the resulting decisions. The ontology therefore provides a common foundation for

Of 799 B0 claims rejected by the frozen contract, 499 are grounded under the new proposition protocol. This includes 333 correctly time-labelled background observations. Other disagreements arise when dependency or citation restrictions reject correct numerical content. No claim that passed the old contract becomes numerically ungrounded under the new scoring. A wrong target attribution still fails because matching uses the asserted interval and independently resolved version.

Queries about source windows and known versions agree with the independent log oracle on all 1201 feature witnesses. Queries about comparison operands agree on 18 eligible displayed comparisons. These checks recover admissible witnesses without requiring a unique gold reasoning graph. The separate prose audit and original auxiliary-verifier disagreements remain available. The bounded scores do not certify unrestricted interpretation.

Withdrawal and fresh generation. An offline comparison uses archived fresh checked answers to the same later-time query as a regeneration proxy (Appendix A). Across all 480 M1 transitions, subject-mean recall rises from 46.5, 44.4 and 41.2% after withdrawal to 76.4, 75.0 and 75.0% for synthetic, WESAD and PPG-DaLiA respectively. Only 84, 80 and 80 of 150 answerable questions per source are complete afterward. Fresh generation improves coverage but does not establish utility over a deterministic template.

5.3 Structural Variation Tests the Maintenance Condition

Table 4. Common-candidate structural replay. Counts aggregate four update conditions per case, separately for exact and corrected-admission generated programs. A required change is independently determined before scheduling. Correction intervals average episodes within subjects identified separately for each dataset. B1 matches B2 and B3 matches M1 in every cell. Missing generated roots are included in the depth/coverage analysis, not invented as initially displayed claims.

Program	Method	Required	Missed	Collateral	Corrected % [CI]
Exact	B0	1260	1260	0	0.0
Exact	B2	1260	720	0	42.9
Exact	M1	1260	0	0	100.0
Generated	B0	1294	1294	0	0.0
Generated	B2	1294	208	0	83.9 [83.0, 84.7]
Generated	M1	1294	0	0	100.0

Across 240 cases and 9,600 replays, full maintenance corrects every required change without withdrawing unaffected claims. Direct maintenance misses 720 of 1260 required changes in exact programs and 208 of 1294 in generated programs. Nonzero exposure occurs in 208/450 eligible corrected generated groups (208/960

automated maintenance and human inspection, with each displayed claim linked to a checkable evidence history.

The real data investigation demonstrates this approach on WESAD laboratory recordings and PPG-DaLiA daily activity recordings. Checked displays achieve 100% source grounding in both datasets, while covering 75.8% and 75.5% of required facts, respectively. The network examples make these results concrete. A corrected WESAD measurement identifies the numerical statements that must be withdrawn. In PPG-DaLiA, a separate recording window preserves a valid conclusion after its primary support fails. These cases show how the representation supports precise decisions about what changes and what remains justified.

The theoretical and controlled results explain when this capability requires following indirect dependencies. Across the controlled generated programs, full maintenance corrects every required change without withdrawing unaffected claims. Direct maintenance misses 183 of 1210 required changes. Transitive exposure predicts the observed misses in all 860 eligible groups, and Neo4j agrees with the relational control on matched updates. The waveform cases also identify when direct checking is sufficient. This combination provides both a working method and an explanation of the conditions under which it adds value.

The present evidence establishes reliable maintenance of bounded numerical claims under controlled revisions of synthetic and real recordings. Complete replacement generation, broader models and evidence discovery, and measured benefits for human reviewers are the next development targets. The central contribution is a tested connection between generated language, explicit support rules and graph-based revision, making the basis of an answer available for both automated checking and visual review.

Use of generative AI. AI assistant supported implementation, and paper language usage. Qwen3-8B generated candidates, and Qwen2.5-3B-Instruct supplied the auxiliary audit.

overall), versus 330/450 eligible exact groups. These controlled patterns test the maintenance condition, not a structure-only prediction. Neo4j and the relational control agree across all 1,920 matched groups.

Alternative support preserves all 120 exact and 111 generated two-route answers after the primary route fails (Figure 3). Single-route answers lose support. Generated answers reach depth one in 74 cases and depth two in 155, while 11 cases yield no admitted answer. None reaches depth three or four, so exact programs provide the deeper tests. The effect depends on which inputs an answer requires and whether another valid route survives, not simply its depth or storage size.

The superseded original admission rules gave 183/1,210 missed changes; those results remain in the archive rather than the primary comparison. Independent truth-table checks assess whether declared support is sufficient and complete.

Exhaustive rechecking baseline. Rechecking every claim matches selective evaluation in all 960 corrected-generated groups (Appendix A). Selective evaluation saves 44.7% of pooled checks, or 26.2% excluding irrelevant updates, but takes 3.25 versus 2.68 microseconds on relevant updates. These CPU kernel timings establish no database performance advantage.

6 Discussion and Conclusion

This study demonstrates revision of bounded numerical claims with explicit evidence dependencies. The method connects generated statements to their measurements, supporting claims and evidence versions in Neo4j. These connections guide revision when evidence changes and retain a checkable evidence history.

Admission improves numerical grounding at the cost of required-fact recall under a shared numerical contract.

The standard maintenance properties and controlled results describe when this capability requires following indirect dependencies. Across the controlled generated programs, full maintenance corrects every required change without withdrawing unaffected claims. Direct-only checking misses updates in the constructed structures; ordinary task-guided generation has zero exposure in all 360 batches. Neo4j agrees with the relational control on matched updates, so a graph store is not necessary for correctness. Exhaustive checking is a reasonable baseline at this small scale; no database speedup is established.

The present evidence establishes reliable maintenance of bounded numerical claims under controlled revisions of synthetic and real recordings. Fresh generation still leaves incomplete answers in the offline comparison. Utility beyond a deterministic template, broader models, evidence discovery, and measured benefits for human reviewers remain development targets.

Use of generative AI. AI assistants supported implementation, analysis tooling and manuscript revision. Qwen3-8B generated candidates, and Qwen2.5-3B-Instruct supplied the auxiliary audit.

References

1. Doyle, J.: A truth maintenance system. Artificial Intelligence **12**(3), 231–272 (1979). https://doi.org/10.1016/0004-3702(79)90008-0
2. Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Metropolitansky, D., Ness, R.O., Larson, J.: From local to global: A graph RAG approach to query-focused summarization (2025), `https://arxiv.org/abs/2404.16130v2`, version 2
3. Green, T.J., Karvounarakis, G., Tannen, V.: Provenance semirings. In: Proceedings of ACM PODS. pp. 31–40 (2007). https://doi.org/10.1145/1265530.1265535
4. Gupta, A., Mumick, I.S., Subrahmanian, V.S.: Maintaining views incrementally. In: Proceedings of ACM SIGMOD. pp. 157–166 (1993). https://doi.org/10.1145/170035.170066
5. Qwen Team: Qwen3-8B model card (2025), `https://huggingface.co/Qwen/Qwen3-8B`, revision b968826d9c46dd6066d109eabc6255188de91218
6. Rasmussen, P., Paliychuk, P., Beauvais, T., Ryan, J., Chalef, D.: Zep: A temporal knowledge graph architecture for agent memory (2025), `https://arxiv.org/abs/2501.13956`
7. Reiss, A., Indlekofer, I., Schmidt, P.: PPG-DaLiA. UCI Machine Learning Repository (2019). https://doi.org/10.24432/C53890
8. Reiss, A., Indlekofer, I., Schmidt, P., Van Laerhoven, K.: Deep PPG: Large-scale heart rate estimation with convolutional neural networks. Sensors **19**(14), 3079 (2019). https://doi.org/10.3390/s19143079
9. Schmidt, P., Reiss, A.: WESAD (wearable stress and affect detection). UCI Machine Learning Repository (2018). https://doi.org/10.24432/C57K5T
10. Schmidt, P., Reiss, A., Duerichen, R., Marberger, C., Van Laerhoven, K.: Introducing WESAD, a multimodal dataset for wearable stress and affect detection. In: Proceedings of the International Conference on Multimodal Interaction. pp. 400–408 (2018). https://doi.org/10.1145/3242969.3242985
11. Turpin, M., Michael, J., Perez, E., Bowman, S.R.: Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting. In: Advances in Neural Information Processing Systems (2023), `https://arxiv.org/abs/2305.04388`
12. vLLM Contributors: Structured outputs (2025), `https://docs.vllm.ai/en/v0.10.2/features/structured_outputs.html`, version 0.10.2
13. W3C PROV Working Group: PROV-DM: The PROV data model. W3C recommendation, World Wide Web Consortium (April 2013), `https://www.w3.org/TR/prov-dm/`

A Additional Baseline Checks

Exhaustive rechecking baseline. A post-review CPU replay compares selective reverse traversal with rechecking every claim, using identical compiled Boolean witnesses. Both match the independent oracle and saved full-maintenance states in all 960 corrected-generated groups. Selective evaluation performs 2,012 versus 3,636 checks (44.7% fewer pooled checks); excluding irrelevant updates, the saving is 26.2% (2,012 versus 2,727). Median per-cell selective/exhaustive times are 2.55/2.66 microseconds overall and 3.25/2.68 on relevant updates. Each cell uses five warm-ups and 31 randomized-order repetitions (Python 3.13.11, Intel Core i5-10400). Timings include cached-state copying, scheduling and AND/OR evaluation; compilation, primitive evaluation, database I/O and generation are excluded. Saved rechecks therefore do not establish a runtime benefit on affected small programs or a database advantage.

Withdrawal and fresh generation. An offline comparison pairs all 480 M1 transitions with the saved fresh checked answer to the identical question at the later knowledge time (160 per source). No new model call or candidate selection is made; this is a proxy for withdraw-and-regenerate, not an online policy test. Subject-mean required recall rises from 46.5% to 76.4% (synthetic), 44.4% to 75.0% (WESAD), and 41.2% to 75.0% (PPG-DaLiA). Paired gains in percentage points, with subject-cluster 95% intervals, are 30.0 [26.8, 32.9], 30.6 [27.8, 33.5], and 33.8 [31.5, 36.0]. Complete answers increase from 37, 37 and 32 to 84, 80 and 80 of 150 answerable questions per source. Fresh answers therefore improve coverage but remain incomplete. They used 246, 251 and 250 archived requests including repairs; median generation times were 10.56, 15.22 and 15.19 seconds on the original RTX 5090. These results do not establish utility over a deterministic template.

Red: removed/replaced. Blue: added/replacement. Page pairing is positional.

No corresponding source page.

References

1. Alchourrón, C.E., Gärdenfors, P., Makinson, D.: On the logic of theory change: Partial meet contraction and revision functions. The Journal of Symbolic Logic **50**(2), 510–530 (1985). https://doi.org/10.2307/2274239
2. Asai, A., Wu, Z., Wang, Y., Sil, A., Hajishirzi, H.: Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In: ICLR (2024). https://arxiv.org/abs/2310.11511
3. Carroll, J.J., Bizer, C., Hayes, P., Stickler, P.: Named graphs. Journal of Web Semantics **3**(4), 247–267 (2005). https://doi.org/10.1016/j.websem.2005.09.001
4. de Kleer, J.: An assumption-based TMS. Artificial Intelligence **28**(2), 127–162 (1986). https://doi.org/10.1016/0004-3702(86)90080-9
5. Dhuliawala, S., et al.: Chain-of-verification reduces hallucination in large language models. In: Findings of ACL. pp. 3563–3578 (2024). https://doi.org/10.18653/v1/2024.findings-acl.212
6. Doyle, J.: A truth maintenance system. Artificial Intelligence **12**(3), 231–272 (1979). https://doi.org/10.1016/0004-3702(79)90008-0
7. Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Metropolitansky, D., Ness, R.O., Larson, J.: From local to global: A graph RAG approach to query-focused summarization (2025), https://arxiv.org/abs/2404.16130v2, version 2
8. Fang, C.M., et al.: PhysioLLM: Supporting personalized health insights with wearables and large language models (2024). https://arxiv.org/abs/2406.19283
9. Gao, T., Yen, H., Yu, J., Chen, D.: Enabling large language models to generate text with citations. In: Proceedings of EMNLP. pp. 6465–6488 (2023). https://doi.org/10.18653/v1/2023.emnlp-main.398
10. Green, T.J., Karvounarakis, G., Tannen, V.: Provenance semirings. In: Proceedings of ACM PODS. pp. 31–40 (2007). https://doi.org/10.1145/1265530.1265535
11. Gupta, A., Mumick, I.S., Subrahmanian, V.S.: Maintaining views incrementally. In: Proceedings of ACM SIGMOD. pp. 157–166 (1993). https://doi.org/10.1145/170035.170066
12. Jensen, C.S., Snodgrass, R.T.: Temporal data management. IEEE Transactions on Knowledge and Data Engineering **11**(1), 36–44 (1999). https://doi.org/10.1109/69.755613
13. Li, Z., Deldari, S., Chen, L., Xue, H., Salim, F.D.: SensorLLM: Aligning large language models with motion sensors for human activity recognition. In: Proceedings of EMNLP. pp. 354–379 (2025). https://doi.org/10.18653/v1/2025.emnlp-main.19
14. Motik, B., Nenov, Y., Piro, R., Horrocks, I.: Incremental update of Datalog materialisation: The backward/forward algorithm. In: Proceedings of AAAI. pp. 1560–1568 (2015). https://doi.org/10.1609/aaai.v29i1.9409
15. Qwen Team: Qwen3-8B model card (2025), https://huggingface.co/Qwen/Qwen3-8B, revision b968826d9c46dd6066d109eabc6255188de91218
16. Rashkin, H., et al.: Measuring attribution in natural language generation models. Computational Linguistics **49**(4), 777–840 (2023). https://doi.org/10.1162/coli_a_00486
17. Rasmussen, P., Paliychuk, P., Beauvais, T., Ryan, J., Chalef, D.: Zep: A temporal knowledge graph architecture for agent memory (2025), https://arxiv.org/abs/2501.13956

No corresponding source page.

18. Reiss, A., Indlekofer, I., Schmidt, P.: PPG-DaLiA. UCI Machine Learning Repository (2019). https://doi.org/10.24432/C53890
19. Reiss, A., Indlekofer, I., Schmidt, P., Van Laerhoven, K.: Deep PPG: Large-scale heart rate estimation with convolutional neural networks. Sensors **19**(14), 3079 (2019). https://doi.org/10.3390/s19143079
20. Schmidt, P., Reiss, A.: WESAD (wearable stress and affect detection). UCI Machine Learning Repository (2018). https://doi.org/10.24432/C57K5T
21. Schmidt, P., Reiss, A., Duerichen, R., Marberger, C., Van Laerhoven, K.: Introducing WESAD, a multimodal dataset for wearable stress and affect detection. In: Proceedings of the International Conference on Multimodal Interaction. pp. 400–408 (2018). https://doi.org/10.1145/3242969.3242985
22. Turpin, M., Michael, J., Perez, E., Bowman, S.R.: Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting. In: Advances in Neural Information Processing Systems (2023), https://arxiv.org/abs/2305.04388
23. vLLM Contributors: Structured outputs (2025), https://docs.vllm.ai/en/v0.10.2/features/structured_outputs.html, version 0.10.2
24. W3C PROV Working Group: PROV-DM: The PROV data model. W3C recommendation, World Wide Web Consortium (April 2013), https://www.w3.org/TR/prov-dm/
25. Wiseman, S., Shieber, S.M., Rush, A.M.: Challenges in data-to-document generation. In: Proceedings of EMNLP. pp. 2253–2263 (2017). https://doi.org/10.18653/v1/D17-1239